Trabalho 2: mestre/escravo com MPI

Bernardo Klein Heitz github.com/bengo501/t2-mpi-smallpt

1. Modelagem

O problema é o path tracer do T1 (smallpt, Kevin Beason, 2008, reescrito em C11). Cada pixel é independente. Uma tarefa é um bloco de G linhas da imagem; G vem de -g (src/smallpt_mpi.c:59). Com 240 linhas, G = 1 dá 240 tarefas, bem mais que os 63 escravos.

O processo 0 não renderiza. Cada escravo começa com TAG_PEDIDO (linha 257). O mestre recebe com MPI_ANY_SOURCE (linha 224) e responde com TAG_TAREFA (y0, número de linhas e o id) ou, se o saco acabou, com TAG_FIM (linha 244), tag própria. O resultado (TAG_RESULTADO) já vale como o próximo pedido: o mestre, no mesmo laço, manda outra tarefa ou o fim (linhas 220-246).

A ordem não segue a chegada. Como cada escravo segura no máximo uma tarefa, task_of[rank] guarda o id entregue e o bloco volta para a linha de origem (linhas 228-242). O último bloco pode ser menor que G, e o número de tarefas não precisa ser múltiplo de P. As tags estão nas linhas 33-41.

Na escalabilidade fraca a mesma imagem é repetida --copies = P vezes. Cada cópia usa a mesma semente, então o custo de uma tarefa não muda e o número de tarefas cresce com P. O checksum impresso é o da primeira cópia e coincidiu com o sequencial em todas as 140 execuções paralelas (copies_ok = 1).

2. Ambiente e metodologia

Cluster Atlântica, nós atlantica03 e atlantica04, sbatch --exclusive, sempre esses dois nós. srun -N 2 --distribution=block: o rank 0 fica em atlantica03 e os demais se dividem meio a meio. Em P = 32 há 15 escravos em atlantica03 e 16 em atlantica04.

lscpu no nó de execução: Xeon E5520, 2 soquetes, 4 núcleos por soquete, 2 threads. São 8 núcleos físicos e 16 threads por nó, não os 8 núcleos por processador do enunciado. Nos dois nós: 16 núcleos físicos e 32 threads. P = 16 coloca um processo por núcleo físico; P = 32 ocupa o hyper-threading; P = 64 coloca dois processos por thread. A partição LAD_geral tem OverSubscribe=NO e recusou --oversubscribe acima de 32 CPUs; P = 64 usou srun -O (overcommit), no mesmo par de nós.

Compilador gcc, flags iguais nas duas versões: -std=c11 -O3 -ffast-math. O núcleo numérico é um único objeto ligado aos dois binários, para o checksum não mudar com flags do mpicc. O sequencial não usa MPI. Entrada de referência: 320 x 240, 64 amostras por subpixel. T1, mediana de 10 rodadas, é 48,03 s (mín. 47,95, máx. 48,20), dentro de 30 a 60 s.

O relógio é MPI_Wtime no mestre, depois da barreira e da alocação, até o último resultado (linhas 216-218 e 248). Câmera, malloc e checksum ficam de fora, nos dois programas. Dez repetições; as tabelas trazem mediana, mínimo e máximo. O trabalho da fraca é o número de raios primários, 4 x largura x altura x spp x copies, e cresceu exatamente com P.

3. Granularidade e balanceamento

Em P = 32 e G = 1, uma tarefa (uma linha do meio da imagem) leva 0,263 s. A ida e volta de uma mensagem do mesmo tamanho (7680 bytes) leva 0,080 ms no mesmo nó e 0,353 ms no outro nó (medianas de 10 sondas; mín. e máx. entre 0,075 e 0,087 ms no local e 0,339 e 0,367 ms no remoto). A razão computação/comunicação é cerca de 3300 no nó local e 750 entre os nós. A comunicação não paga o custo de juntar várias linhas.

Por isso a comparação em P = 32 favorece G = 1. G = 4 fica empatado (2,61 s contra 2,60 s). G = 16 piora para 5,27 s: são só 15 tarefas para 31 escravos, e metade termina sem trabalho. Não há granularidade menor que uma linha sem quebrar a semente por linha, que é o que torna o pixel independente de quem o calcula.

Na primeira rodada com P = 32 e G = 1, os escravos fizeram de 7 a 9 tarefas (soma 240) e ficaram ocupados entre 2,28 s e 2,60 s. O tempo de parede, 2,60 s, é o do escravo mais lento: quem fecha a execução é a cauda, não uma fila no mestre. 240 mensagens a 0,35 ms dariam 0,08 s, perto de 3% desses 2,60 s.

Tabela 1. Granularidade em P = 32, mesma entrada da forte.

G	med. (s)	min	max
1	2,60	2,60	2,61
4	2,61	2,60	2,62
16	5,27	5,26	5,37

4. Análise

O teto da forte, antes de medir, é o número de escravos que cabem em núcleo físico: 16 núcleos nos dois nós, um deles com o mestre, logo 15 escravos. P = 2 tem um único escravo, ainda por cima no outro nó, e o speed-up fica em 0,99, não em 2. O acréscimo sobre T1 é de 0,35 s (menos de 1%): a rede não explica o speed-up perto de 1; explica-o o fato de só um processo calcular. A eficiência pedida, S/P, dá 0,50 porque conta o mestre parado. A eficiência por escravo, S/(P-1), dá 0,99 e descreve o uso de quem trabalha.

De P = 4 a P = 16 o speed-up acompanha os escravos: 2,97, 6,82 e 14,03. Em P = 16 há 15 escravos e 16 núcleos físicos; S = 14,03 é 0,94 do teto de 15. A folga de 6% cabe na cauda (as linhas do centro da cena custam mais) e nos 0,08 s de mensagens.

P = 32 não dobra P = 16. O speed-up vai de 14,03 para 18,46, ganho de 1,32, compatível com hyper-threading em código limitado por unidade de ponto flutuante, não com um segundo núcleo. P = 64, dois processos por thread, cai para 17,17: overcommit disputa o mesmo núcleo e piora. O mestre não é o gargalo. Se fosse, o tempo pararia de cair já com poucas dezenas de tarefas; em P = 16 o tempo ainda cai quase na proporção dos escravos.

Trabalho 2: mestre/escravo com MPI

Bernardo Klein Heitz github.com/bengo501/t2-mpi-smallpt

A fraca usa a mesma entrada e multiplica as cópias por P. Os raios primários passam de 1,97e7 para exatamente P vezes isso. A eficiência T1/Tp fica em 0,50, 0,74, 0,86, 0,88, 0,60 e 0,30. Até P = 16 ela acompanha a eficiência forte (diferença de no máximo 0,02). Isso é o que se espera quando a comunicação é uma fração pequena e constante do trabalho: fazer P vezes mais tarefas com o mesmo speed-up leva cerca de P/S vezes T1, e T1/Tp vira S/P. A fraca não se sustenta melhor que a forte. As duas quebram no mesmo lugar, o HT, porque o limite é o núcleo, não a falta de tarefas. Em P = 32 cada escravo ainda recebe cerca de 8 linhas na forte e cerca de 250 na fraca; a cauda relativa é menor na fraca, e por isso 0,60 fica um pouco acima de 0,58, não o contrário.

Entre nós, o efeito aparece na sonda (0,35 ms contra 0,08 ms) e some no tempo total. Com block, a partir de P = 4 há escravos nos dois nós, mas 0,08 s de rede não movem uma curva cujo degrau grande é o de 14 para 18 ao ligar o HT.

5. Uso de ferramentas de IA

O código MPI, os scripts da campanha e a redação deste relatório foram feitos com o Cursor (modelo Grok 4.7), em cima do smallpt do T1. A origem do algoritmo é Beason (2008). As 10 repetições, o checksum e os tempos brutos saíram do cluster, não do modelo. O repositório tem o fonte, o Makefile, o sbatch, results/runs.csv e results/raw/.

Referencia

K. Beason. smallpt: Global Illumination in 99 lines. 2008. kevinbeason.com/smallpt/

Tabela 2. Escalabilidade forte. T1 = 48,03 s. S = T1/Tp, E = S/P.

P	med. (s)	min	max	S	E
2	48,38	48,21	48,57	0,99	0,50
4	16,15	16,11	16,25	2,97	0,74
8	7,04	7,02	7,05	6,82	0,85
16	3,42	3,42	3,61	14,03	0,88
32	2,60	2,60	2,61	18,46	0,58
64	2,80	2,73	2,90	17,17	0,27

Tabela 3. Escalabilidade fraca. E = T1/Tp. Raios = P x 1,97e7.

P	med. (s)	min	max	E
2	96,92	96,43	97,14	0,50
4	64,57	64,42	64,79	0,74
8	55,55	55,38	55,80	0,86
16	54,47	54,41	54,58	0,88
32	80,17	79,96	80,32	0,60
64	159,42	159,35	159,62	0,30

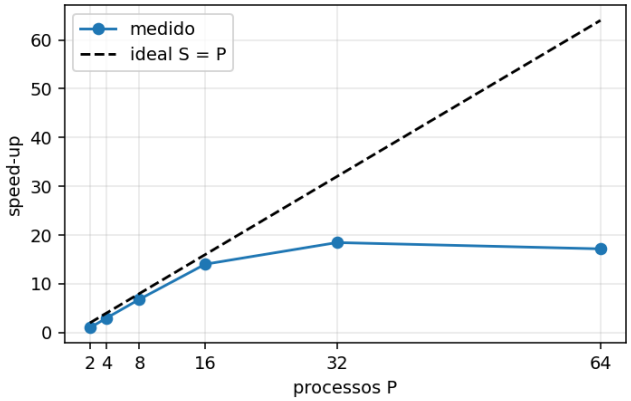


Figura 1. Speed-up da escalabilidade forte e a reta S = P.

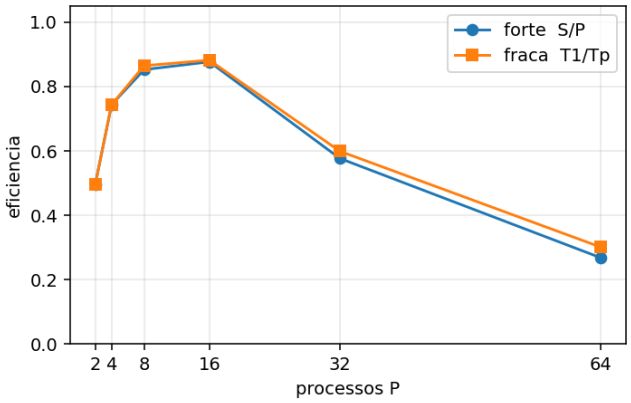


Figura 2. Eficiência forte (S/P) e fraca (T1/Tp).